S04 – BigQuery Avanzado y Analytics Engineering

Evidencias de ejecución – IT Academy, Data Analytics

Nombre	*Carlos Hisamitsu*
P2P	*Gabriela*
Fecha	*21/10/2026*
Repositorio GitHub	https://github.com/chisamitsu/ita-dataanalytics/tree/main/s04-bigquery-avanzado

NIVEL 1: Entorno e ingesta híbrida (Code-First)

Ejercicio 1 – Diagnóstico sobre tabla no optimizada

Enunciado

Consulta que cruza transacciones y compañías filtrando por el 12 de marzo de 2022 y país “Germany”. Dry run para comprobar que, sin partición, BigQuery hace un full table scan aunque se pida solo un día.

Evidencia de ejecución

Consulta y resultado del Dry Run (bytes procesados)

```sql
SELECT
  transactions.transaction_id,
  transactions.timestamp,
  transactions.amount,
  company.company_name,
  company.country
FROM `sprint3-analytics-hisamitsu.sprint3_silver.transactions_clean` AS transactions
INNER JOIN `sprint3-analytics-hisamitsu.sprint3_silver.companies_clean` AS company
  ON transactions.business_id = company.company_id
WHERE DATE(transactions.timestamp) = '2022-03-12'
  AND company.country = 'Germany';
```

This query will process 6.68 MB when run.

```sql
SELECT
  transactions.transaction_id,
  transactions.timestamp,
  transactions.amount,
  company.company_name,
  company.country
FROM `sprint3-analytics-hisamitsu.sprint3_silver.transactions_clean` AS transactions
INNER JOIN `sprint3-analytics-hisamitsu.sprint3_silver.companies_clean` AS company
  ON transactions.business_id = company.company_id;
```

This query will process 6.68 MB when run.

Comentarios sobre los resultados

El dryrun informa que la query procesa 6.68MB con o sin el filtro (WHERE).

Ejercicio 2 – Re-arquitectura del almacenamiento (Partition & Cluster)

Enunciado

En dos pasos: (1) generar transactions_recent a partir de transactions_clean con el timestamp desplazado a los últimos 50 días (para evitar la caducidad de particiones del sandbox); (2) crear fact_transactions_optimized particionada por DATE(timestamp) y clusterizada por business_id.

Paso 1: Generación de datos recientes

Creación de transactions_recent

```sql
CREATE OR REPLACE TABLE `sprint3-analytics-hisamitsu.sprint3_silver.transactions_recent` AS
SELECT
  * EXCEPT(timestamp),
  TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST(RAND() * 50 AS INT64) DAY) AS timestamp
FROM `sprint3-analytics-hisamitsu.sprint3_silver.transactions_clean`;
```

Query completed

Using on-demand processing quota

Query results

Job information | Results | Execution details | Execution graph

This statement created a new table named transactions_recent.

Paso 2: Tabla particionada y clusterizada

DDL de fact_transactions_optimized

```sql
CREATE OR REPLACE TABLE `sprint3-analytics-hisamitsu.sprint3_gold.fact_transactions_optimized`
PARTITION BY DATE(timestamp)
CLUSTER BY business_id AS
  SELECT *
  FROM `sprint3-analytics-hisamitsu.sprint3_silver.transactions_recent`;
```

Query completed

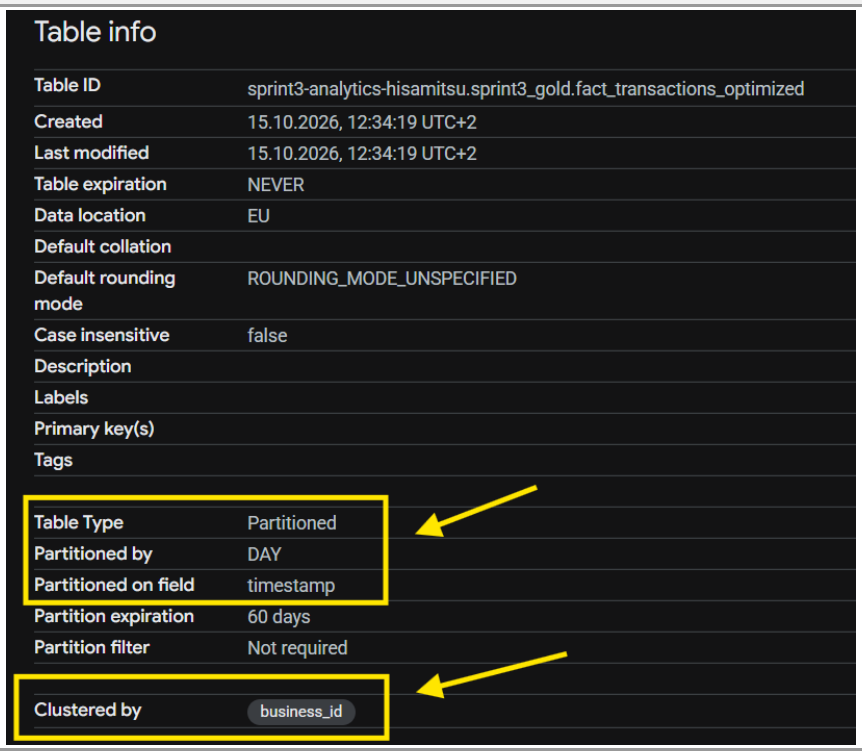
Using on-demand processing quota

Query results

Job information | Results | Execution details | Execution graph

This statement created a new table named fact_transactions_optimized.

Detalles de la tabla: partición y clustering



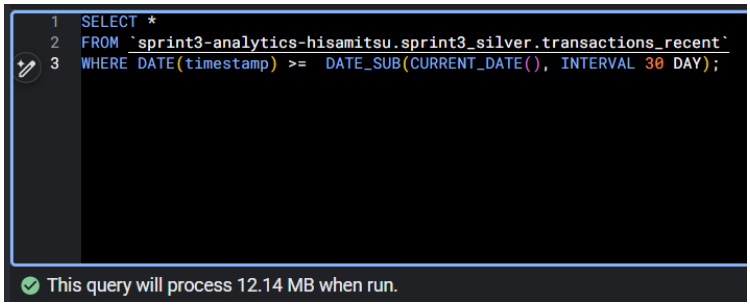
Ejercicio 3 – Benchmark de coste (Dry Run)

Enunciado

Misma consulta (SELECT * de los últimos 30 días) apuntada primero a transactions_recent y después a fact_transactions_optimized, comparando los bytes procesados que muestra el Dry Run en cada caso.

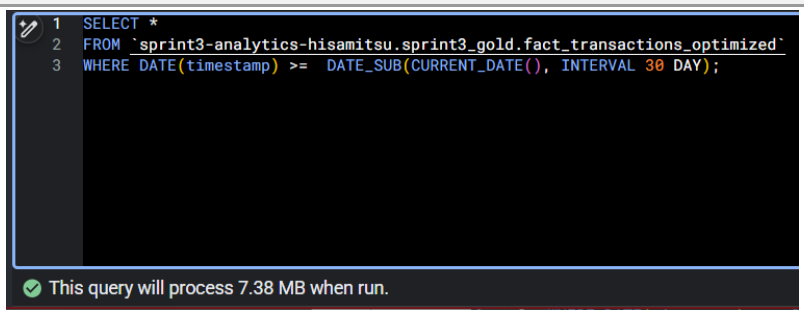
Paso 1: Tabla no optimizada

Dry Run sobre transactions_recent



Paso 2: Tabla optimizada

Dry Run sobre fact_transactions_optimized



Comentarios sobre los resultados

Con el dryrun podemos ver que la consulta en la tabla optimizada tiene un coste aproximado de 40% mas bajo que la misma consulta en la tabla no optimizada (de 12.14 MB a 7.38 MB). Sabemos que los datos aleatorios que generamos están distribuidos en los últimos 50 días por lo que tiene sentido que sea más o menos esa la diferencia si filtramos los datos de los últimos 30 días.

Ejercicio 4 – Smart Caching (Vista Materializada)

Enunciado

Vista materializada mv_daily_sales con las ventas totales por día, para que el dashboard no recalcule la agregación en cada refresco.

Evidencia de ejecución

Creación de mv_daily_sales

```
CREATE OR REPLACE MATERIALIZED VIEW `sprint3-analytics-hisamitsu.sprint3_gold.mv_daily_sales` AS
SELECT DATE(timestamp) AS sales_date,
  SUM(amount) AS total_sales
FROM `sprint3-analytics-hisamitsu.sprint3_gold.fact_transactions_optimized`
WHERE declined = '0'
GROUP BY sales_date;
```

This query will process 0 B when run.

Query results

Job information | Results | Execution details | Execution graph

This statement created a new materialized view named mv_daily_sales.

Materialized view info

Materialized view ID	sprint3-analytics-hisamitsu.sprint3_gold.mv_daily_sales
Created	15.10.2026, 13:11:04 UTC+2
Last modified	15.10.2026, 13:11:07 UTC+2
Materialized view expiration	14.12.2026, 12:11:04 UTC+1
Last refresh time	15.10.2026, 13:11:04 UTC+2
Refresh enabled	true
Refresh interval (ms)	1800000
Max staleness	

Consulta mv_daily_sales (bytes procesados)

```
SELECT *
FROM `sprint3-analytics-hisamitsu.sprint3_gold.mv_daily_sales`
ORDER BY sales_date DESC;
```

Query completed

Using on-demand processing quota

Query results

Job information | Results | Visualization | JSON | Executi

Job ID	sprint3-analytics-hisamitsu:EU.job_ZYXQOyXKnDGr
User	c.hisamitsu@gmail.com
Location	EU
Creation time	15.10.2026, 13:15:03 UTC+2
Start time	15.10.2026, 13:15:03 UTC+2
End time	15.10.2026, 13:15:03 UTC+2
Duration	315 ms
Bytes processed	1.2 KB
Bytes billed	10 MB
Slot milliseconds	49

Query results

Job information | Results | Visua

Row	sales_date	total_sales
1	2026-10-15	276749.68
2	2026-10-14	531694.36
3	2026-10-13	521238.83
4	2026-10-12	509533.81
5	2026-10-11	499600.99
6	2026-10-10	519084.49
7	2026-10-09	512342.03
8	2026-10-08	527647.38
9	2026-10-07	509171
10	2026-10-06	542312.7
11	2026-10-05	508733.2
12	2026-10-04	506675.26
13	2026-10-03	518146.66

Results per page: 50 1 – 50 of 51

Comentarios sobre los resultados

La vista se refresca sola a cada 30 min. Al consultarla proceso 1.2 KB en vez de recalcular la suma sobre fact_transactions_optimized cada vez.

NIVEL 2: SQL Analítico Avanzado

Ejercicio 1 – Perfilado de clientes VIP (CTE)

Enunciado

CTE VIP_Stats que agrupa por usuario (gasto total, número de transacciones, ticket medio, compra máxima), filtrando los que superan 500€ de gasto acumulado, cruzado con users_combined para los datos personales. Ordenado por gasto total descendente.

Evidencia de ejecución

Consulta y resultado de clientes VIP

```sql
WITH VIP_Stats AS
  (SELECT
    user_id,
    SUM(amount) AS total_gastat,
    COUNT(*) AS num_compres,
    ROUND(AVG(amount),2) AS tiquet_mig,
    MAX(amount) AS max_compra
  FROM `sprint3-analytics-hisamitsu.sprint3_silver.transactions_clean`
  WHERE declined = '0'
  GROUP BY user_id
  HAVING total_gastat > 500)
SELECT
  users.user_id,
  CONCAT(users.name, ' ', users.surname) AS nom_complet,
  users.email,
  VIP_Stats.num_compres,
  VIP_Stats.tiquet_mig,
  VIP_Stats.max_compra,
  VIP_Stats.total_gastat
FROM `sprint3-analytics-hisamitsu.sprint3_silver.users_combined` AS users
INNER JOIN VIP_Stats
  ON VIP_Stats.user_id = users.user_id
ORDER BY VIP_Stats.total_gastat DESC;
```

This query will process 2.84 MB when run.

Using on-demand processing quota

Query results | Chat | View all jobs | Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	user_id	nom_complet	email	num_compres	tiquet_mig	max_compra	total_gastat
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449	615.51	38614.39
3	455	Fwfll Phylgnmhnd	fwfll.phylgnmhnd@example.com	66	522.6	724.44	34491.27
4	417	Fqatil Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	73	421.87	675.78	30796.3

Results per page: 50 | 1 – 50 of 5000

Comentarios sobre los resultados

He usado transactions_clean en vez de fact_transactions_optimized. En un caso real con la tabla particionada usaría esa, pero aquí no cambiaría nada pues VIP_Stats agrupa todo el histórico sin filtrar por fecha ni por empresa, así que el particionado y el clustering no tendrían nada que podar.

Además, con el umbral de 500€ todos los 5000 usuarios salen como VIP.

Ejercicio 2 – Análisis de tendencias (Window Functions)

Enunciado

Sobre mv_daily_sales, comparativa día a día con LAG(): ventas de hoy, ventas de ayer y variación porcentual redondeada a 2 decimales.

Evidencia de ejecución

Consulta y resultado con LAG()

```sql
SELECT
  sales_date AS Data,
  total_sales AS Vendes_Avui,
  LAG(total_sales) OVER (ORDER BY sales_date) AS Vendes_Ahir,
  ROUND(
    SAFE_DIVIDE(total_sales - LAG(total_sales) OVER (ORDER BY sales_date),
                LAG(total_sales) OVER (ORDER BY sales_date)) * 100,
    2
  ) AS Diff_Percentual
FROM `sprint3-analytics-hisamitsu.sprint3_gold.mv_daily_sales`
ORDER BY Data;
```

This query will process 1.2 KB when run.

Using on-demand processing quota

Query results — Chat — View all jobs — Save results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	Data	Vendes_Avui	Vendes_Ahir	Diff_Percentual
1	2026-08-26	257542.48	*null*	*null*
2	2026-08-27	526732.85	257542.48	104.52
3	2026-08-28	505642.71	526732.85	-4
4	2026-08-29	509072.59	505642.71	0.68
5	2026-08-30	527264.67	509072.59	3.57
6	2026-08-31	530335.73	527264.67	0.58
7	2026-09-01	514154.85	530335.73	-3.05
8	2026-09-02	520653.54	514154.85	1.26
9	2026-09-03	546359.97	520653.54	4.94
10	2026-09-04	520560.68	546359.97	-4.72
11	2026-09-05	511388.44	520560.68	-1.76
12	2026-09-06	517830.8	511388.44	1.26

Results per page: 50 — 1 – 50 of 51

Comentarios sobre los resultados

No he puesto ROUND en Vendes_Avui ni Vendes_Ahir porque ya vienen de total_sales (NUMERIC con 2 decimales) y no cambiaría nada. Solo hace falta en Diff_Percentual, que sale de una división

Ejercicio 3 – Totales acumulados (Running Total YTD)

Enunciado

Sobre mv_daily_sales, ventas del día y suma acumulada que se reinicia cada 1 de enero (PARTITION BY año, ventana ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), ambas redondeadas a 2 decimales.

Evidencia de ejecución

Consulta y resultado del acumulado YTD

```sql
SELECT
  sales_date AS Data,
  ROUND(total_sales, 2) AS `Vendes del Dia`,
  ROUND(SUM(total_sales) OVER (
    PARTITION BY EXTRACT(YEAR FROM sales_date)
    ORDER BY sales_date ASC
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ), 2) AS `Vendes Acumulades YTD`
FROM `sprint3-analytics-hisamitsu`.sprint3_gold.mv_daily_sales
ORDER BY sales_date;
```

This query will process 1.2 KB when run.

Using on-demand processing quota

Query results

Job information | Results | Visualization | JSON | Executio

Row	Data	Vendes del Dia	Vendes Acumula...
1	2026-08-26	257542.48	257542.48
2	2026-08-27	526732.85	784275.33
3	2026-08-28	505642.71	1289918.04
4	2026-08-29	509072.59	1798990.63
5	2026-08-30	527264.67	2326255.3
6	2026-08-31	530335.73	2856591.03
7	2026-09-01	514154.85	3370745.88
8	2026-09-02	520653.54	3891399.42
9	2026-09-03	546359.97	4437759.39
10	2026-09-04	520560.68	4958320.07
11	2026-09-05	511388.44	5469708.51
12	2026-09-06	517830.8	5987539.31
13	2026-09-07	520339.24	6507878.55

Comentarios sobre los resultados

ROUND no cambia nada en este caso porque la columna total_sales ya es NUMERIC con 2 decimales, lo dejo por pedirlo el enunciado.

Ejercicio 4 – Fidelización y valor del cliente (QUALIFY)

Enunciado

Usuarios que han llegado a su tercera compra: datos de usuario, fecha e importe de esa tercera compra y la media de las tres primeras. ROW_NUMBER() para el orden cronológico y QUALIFY para filtrar, acotando las filas antes de agregar.

Evidencia de ejecución

Consulta y resultado del listado de fidelización

```sql
WITH numbered_transactions  AS (
  SELECT
    user_id,
    timestamp,
    amount,
    ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp) AS row_num
  FROM `sprint3-analytics-hisamitsu`.sprint3_silver.transactions_clean
  WHERE declined = '0'
  QUALIFY row_num <= 3
)
SELECT
  transactions.user_id,
  CONCAT(users.name, ' ', users.surname) AS nom_complet,
  users.email,
  MAX(IF(transactions.row_num = 3, transactions.timestamp, NULL)) AS `data_3a_compra`,
  MAX(IF(transactions.row_num = 3, transactions.amount, NULL)) AS `import_3a_compra`,
  ROUND(AVG(transactions.amount), 2) AS import_mitjana_3_compres
FROM numbered_transactions AS transactions
INNER JOIN `sprint3-analytics-hisamitsu`.sprint3_silver.users_combined AS users
  ON transactions.user_id = users.user_id
GROUP BY transactions.user_id, nom_complet, email
HAVING COUNT(*) = 3
ORDER BY transactions.user_id;
```

Query completed

Using on-demand processing quota

Query results

Chat | View all jobs | Save results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	user_id	nom_complet	email	data_3a_compra	import_3a_compra	import_mitjana_3...
1	1	Zeus Gamble	interdum.enim@protonmail.edu	2015-12-20 18:06:59 UTC	161.33	239.13
2	2	Garrett Mcconnell	integer.vitae.nibh@protonmail.o...	2015-09-21 01:23:22 UTC	267.6	287.46
3	3	Ciaran Harrison	interdum.feugiat@aol.org	2016-01-22 17:27:05 UTC	59.76	166.44
4	4	Howard Stafford	ornare.egestas@icloud.edu	2016-09-28 16:35:32 UTC	171.13	225.76
5	5	Hayfa Pierce	et.malesuada.fames@hotmail.o	2015-08-17 21:34:14 UTC	51.94	184.49

Results per page: 50 | 1 – 50 of 5000

Comentarios sobre los resultados

Salen todos los 5000 usuarios. En este dataset todo el mundo ya tiene sus 3 compras, así que el HAVING no filtra a nadie. En un caso real limitaría por fecha (por ejemplo, solo en los últimos 12 meses), para que la fidelización no dependa simplemente de cuánto tiempo lleva el cliente.

NIVEL 3: Analytics Engineering (Arrays & Automatización)

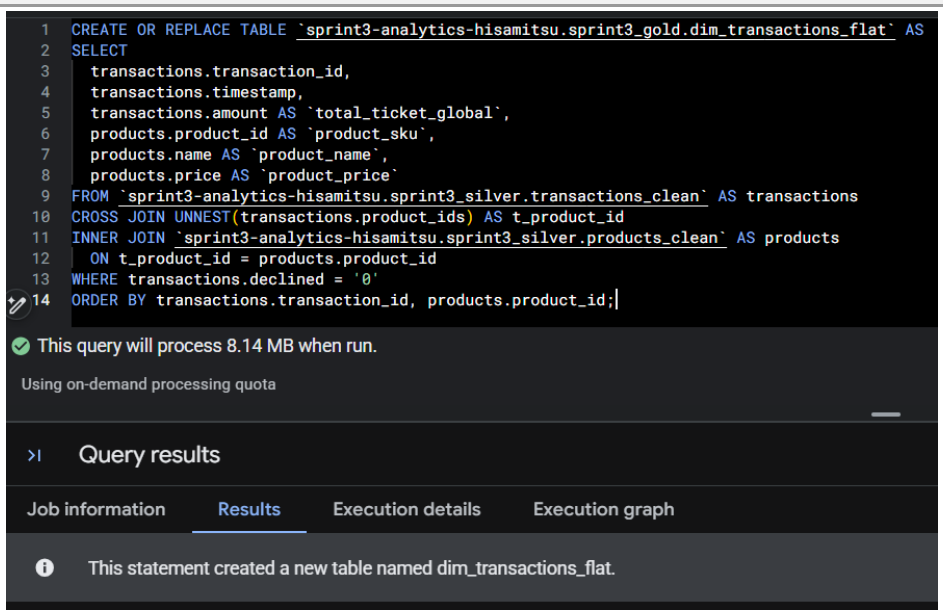
Ejercicio 1 – Desanidado y aplanado de datos (Unnesting)

Enunciado

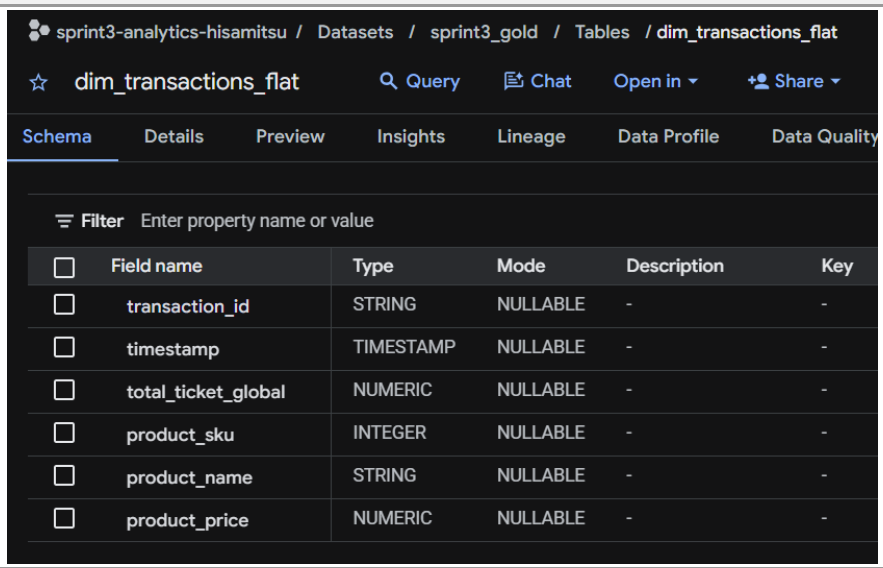
Tabla dim_transactions_flat: CROSS JOIN UNNEST(product_ids) de transactions_clean cruzado con products_clean para obtener nombre y precio de cada producto, una fila por producto vendido (sin agrupar).

Evidencia de ejecución

Vista previa de dim_transactions_flat



Esquema de dim_transactions_flat



Comentarios sobre los resultados

El ejercicio pide que una columna se llame “total_ticket (Global)”, pero los paréntesis no son caracteres válidos (ref.). La he nombrado total_ticket_global para mantener el patrón del resto de columnas. No hay dato real de SKU de productos, así que uso el product_id como valor, aunque la columna sigue llamándose product_sku como pide el enunciado.

Ejercicio 2 – Ranking de ventas (agregación simple)

Enunciado

Top 5 de productos más vendidos, agrupando dim_transactions_flat por nombre de producto y contando filas.

Evidencia de ejecución

Consulta y resultado del Top 5

```sql
WITH product_counts AS (
  SELECT
    product_sku,
    product_name,
    COUNT(*) AS units_sold
  FROM `sprint3-analytics-hisamitsu.sprint3_gold.dim_transactions_flat`
  GROUP BY product_sku, product_name
)
SELECT
  RANK() OVER (ORDER BY units_sold DESC) AS rank,
  product_sku,
  product_name,
  units_sold
FROM product_counts
QUALIFY rank <= 5
ORDER BY units_sold DESC;
```

This query will process 5.97 MB when run.

Using on-demand processing quota

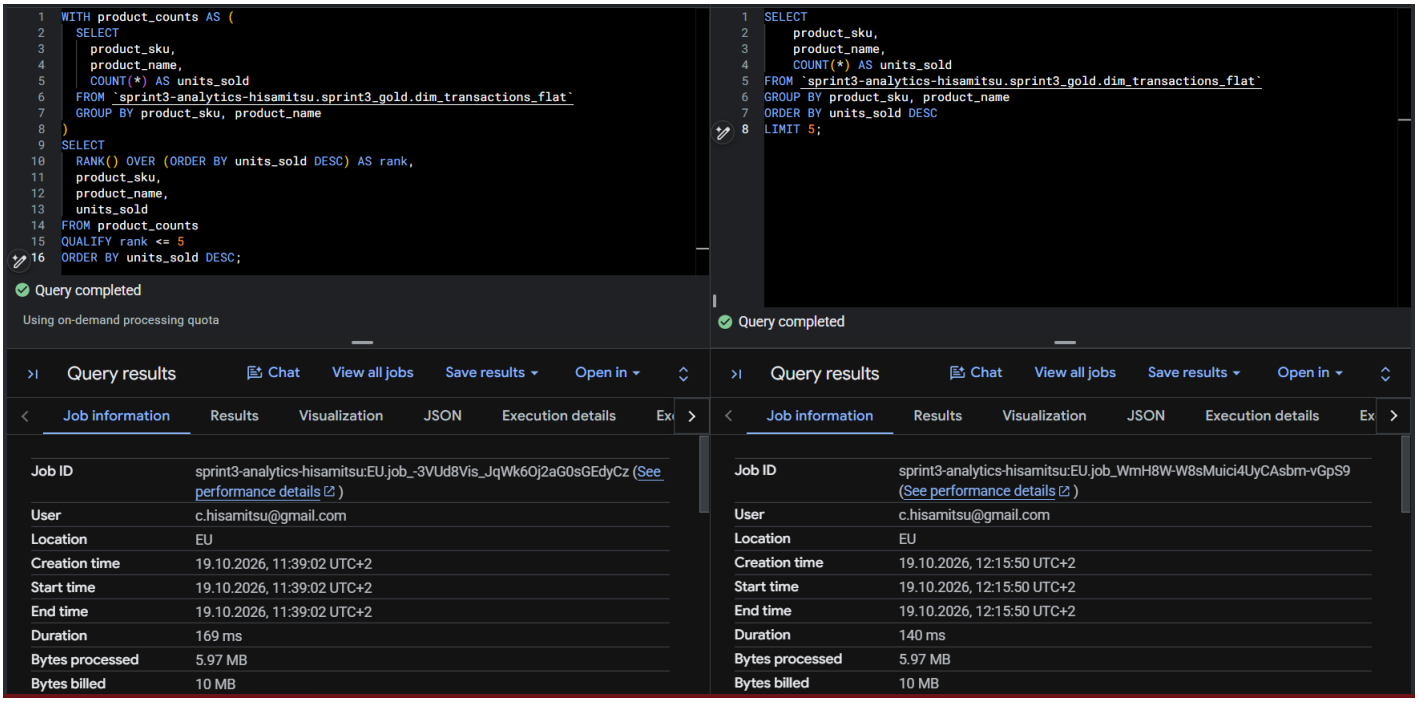
Query results | Chat | View all jobs

Job information | Results | Visualization | JSON | Execution details | Exec

Row	rank	product_sku	product_name	units_sold
1	1	52	riverlands the duel	2642
2	2	29	Tully maester Tarly	2627
3	3	21	duel Direwolf	2603
4	4	16	the duel warden	2602
5	5	33	duel warden	2593

Comentarios sobre los resultados

Ese ejercicio lo he hecho con RANK para entrenar, pero un SELECT con LIMIT 5 traería el mismo resultado. Sería distinto en el caso de haber más productos con el mismo número de ventas en la posición 5.



He comparado las dos formas y el consumo de datos es el mismo, pero el tiempo con LIMIT fue un poco más rápido (aunque con una sola ejecución de cada no es una comparación muy fiable).

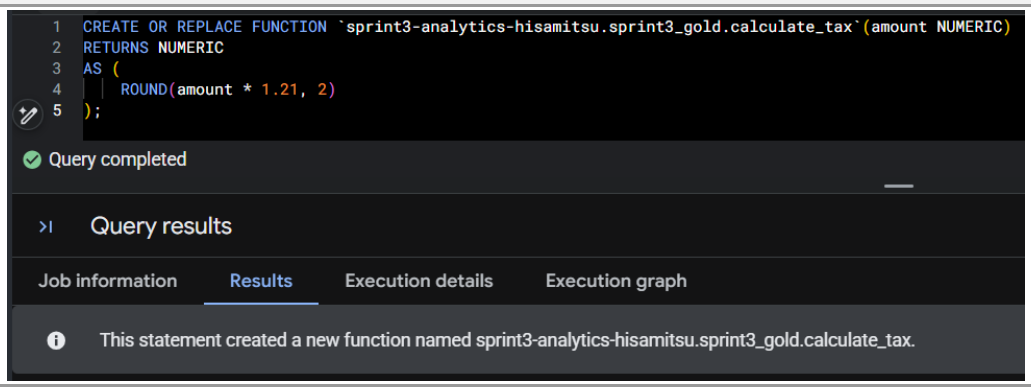
Ejercicio 3 – Automatización del pipeline y visualización

Enunciado

UDF calculate_tax(amount) para no dejar el 21% de IVA harcodeado; dim_transactions_flat ampliada con product_price_tax_inc usando esa función; Scheduled Query diaria a las 07:00 que regenera la tabla; dashboard en Looker Studio conectado a la tabla resultante.

a) Función UDF calculate_tax

Creación de la función calculate_tax



b) Tabla dim_transactions_flat con IVA

DDL actualizado de dim_transactions_flat

```sql
CREATE OR REPLACE TABLE `sprint3-analytics-hisamitsu.sprint3_gold.dim_transactions_flat` AS
SELECT
  transactions.transaction_id,
  transactions.timestamp,
  transactions.amount AS `total_ticket_global`,
  products.product_id AS `product_sku`,
  products.name AS `product_name`,
  products.price AS `product_price`,
  `sprint3-analytics-hisamitsu.sprint3_gold.calculate_tax`(products.price) AS `product_price_tax_inc`
FROM `sprint3-analytics-hisamitsu.sprint3_silver.transactions_clean` AS transactions
CROSS JOIN UNNEST(transactions.product_ids) AS t_product_id
INNER JOIN `sprint3-analytics-hisamitsu.sprint3_silver.products_clean` AS products
  ON t_product_id = products.product_id
WHERE transactions.declined = '0'
ORDER BY transactions.transaction_id, products.product_id;
```

Query completed

Using on-demand processing quota

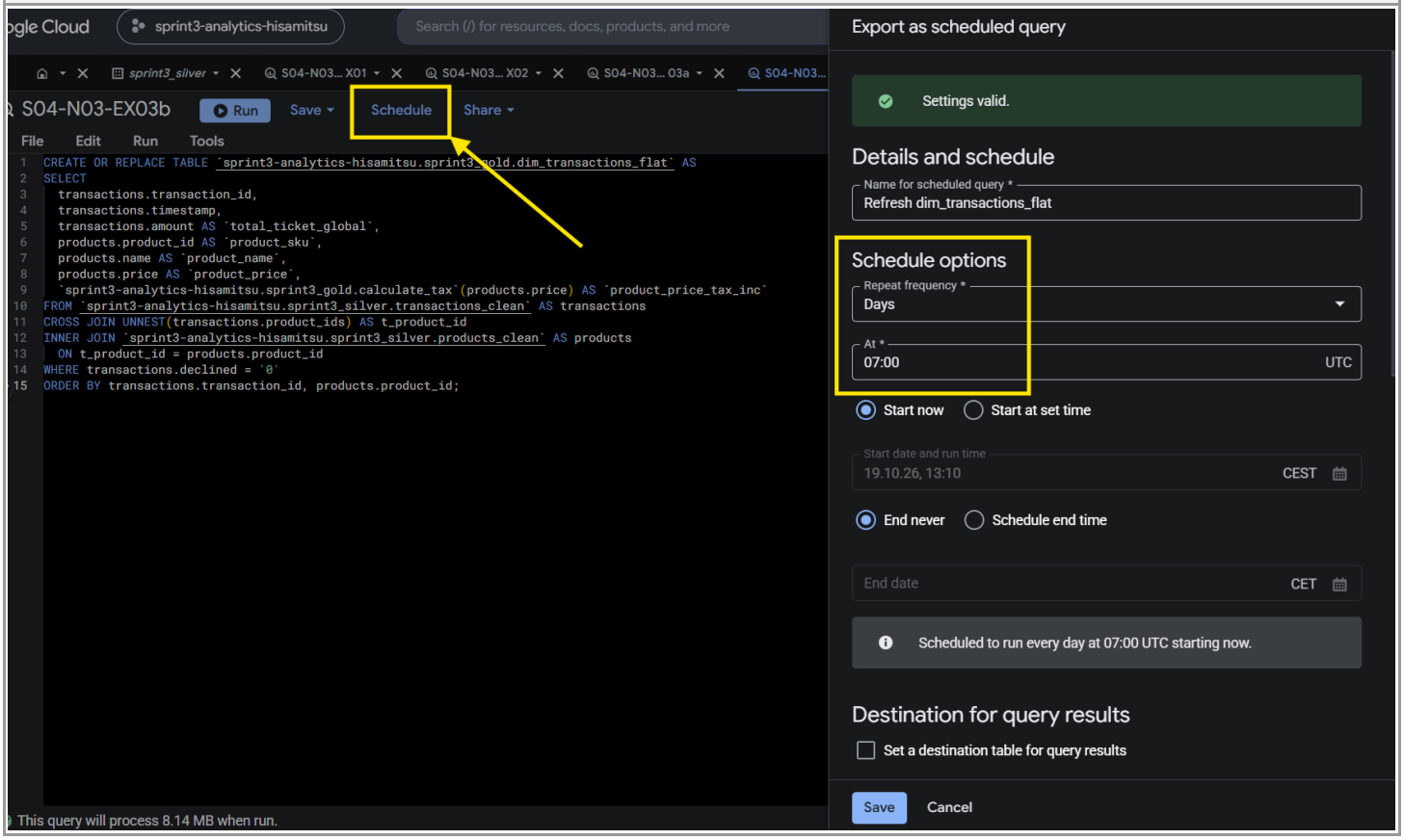
Query results

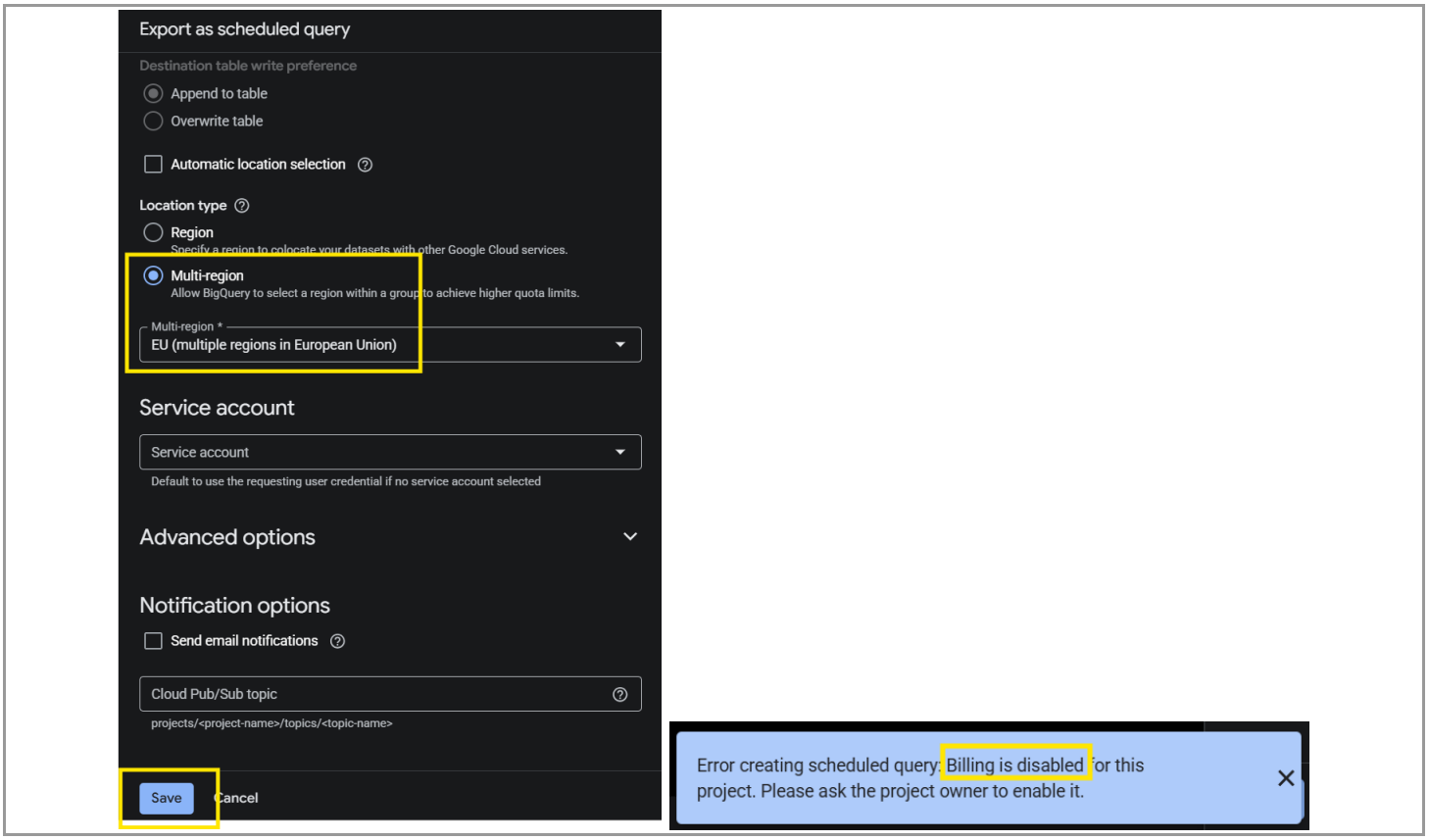
Job information | Results | Execution details | Execution graph

This statement replaced the table named dim_transactions_flat.

c) Scheduled Query

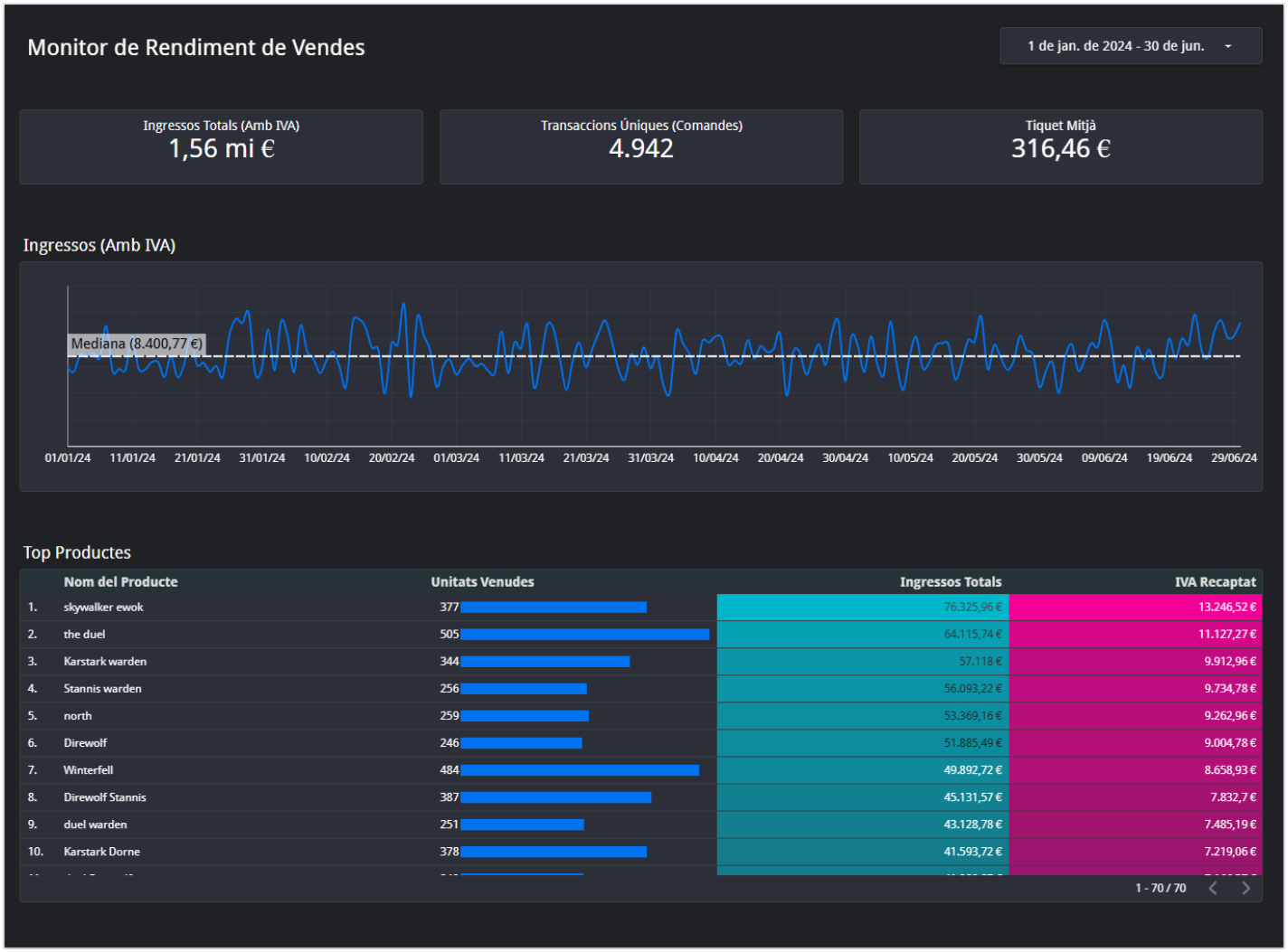
Programación de la Scheduled Query (07:00 AM)





d) Dashboard en Looker Studio

Enlace: https://datastudio.google.com/reporting/f4f4d35b-57f0-4241-9577-624ce4d33bc7



Comentarios sobre los resultados

La scheduled query no me dejó crearla: el proyecto está en el Sandbox sin billing, da error tanto por UI como por bq. Dejo la captura del error.

calculate_tax la dejé con el 21% fijo como pide el enunciado. Se podria hacer más dinámico con una tabla de porcentajes por país y pasando el país como parámetro, pero no lo he hecho.